Behavior-Constrained Agent Systems: A Control-Theoretic View of Safe Authorization

Long LI

lilong@gmail.com

March 11, 2026

ABSTRACT

As software agents gain direct access to tools, APIs, payment rails, deployment pipelines, and persistent credentials, the classical authorization model becomes inadequate. Ordinary signatures and API tokens answer who issued a request, but not whether the request remained inside a pre-committed behavioral boundary. In practice, agent safety therefore relies heavily on software-layer guardrails such as middleware checks, policy engines, prompt instructions, workflow approvals, and sandbox controls. These mechanisms are useful, but they remain bypassable by prompt injection, implementation bugs, runtime replacement, configuration drift, and unsafe execution paths.

This paper builds on the behavior-bound signature direction introduced by **Y.Y.N. Li** in *Behavior-Bound Signatures: Cryptographic Policy Enforcement from Zero-Knowledge Soundness* [1]. We do not propose a new cryptographic construction. Instead, we present a systems and control-theoretic framing for how behavior-constrained authorization should be deployed in practical agent systems. Our central claim is that, for high-risk agent actions, behavior-constrained authorization should be understood not merely as a stronger signature primitive, but as a **negative-feedback control architecture**: actions are structured, authorization is policy-bound, validators become mandatory execution gates, and structured rejection reasons act as error signals that drive safe retry.

We develop this claim through a reference architecture, a related-work positioning, an action-modeling discipline, and application analyses spanning payments, API invocation, autonomous software development, database mutation, file deletion, and enterprise approvals. We further argue that blockchain is not required for the first useful deployments; the immediate value lies in local or organizational systems that constrain delegated authority at the action boundary. The resulting thesis is deliberately narrow: behavior-constrained authorization is most useful today as a hard control layer for high-risk machine actions, not as a replacement for all agent infrastructure.

KEYWORDS: agent safety, behavior-constrained authorization, behavior-bound signatures, control theory, validator gate, policy-bound signing

1. Introduction

Software systems are moving from interactive assistants toward autonomous or semi-autonomous agents that can plan, invoke tools, modify code, trigger deployments, update databases, move funds, and call external APIs. Once such agents are given direct credentials,

the security problem changes. The system is no longer defending only against outsiders who lack the key. It must also defend against delegated executors that possess valid credentials but may act outside their intended behavioral envelope.

This is a practical engineering problem rather than a purely theoretical one. An agent can be prompt-injected, misaligned with the current task, buggy, partially compromised through tools or plugins, replaced by a different runtime, or simply operating under stale context. Under an ordinary authorization model, however, any request signed with a valid key is structurally legitimate. Whether that request should have been authorized is decided only afterward, in surrounding software layers. That separation creates a persistent gap between **identity validity** and **behavioral validity**.

The motivating observation of this paper is straightforward: many risky agent failures are not caused by broken cryptography, but by overpowered credentials. The agent is allowed to sign or invoke too much, and the system relies on non-binding software discipline to compensate. This is exactly the gap that behavior-bound signature ideas, as developed by **Y.Y.N. Li**, are meant to narrow.

1.1 Relation to the Reference Paper

This paper explicitly builds on **Y.Y.N. Li's** reference paper, *Behavior-Bound Signatures: Cryptographic Policy Enforcement from Zero-Knowledge Soundness* [1]. Li's paper provides the core cryptographic intuition: authorization should be coupled to behavioral constraints, rather than treated as a free signature followed by external checking. That contribution is foundational.

Our contribution is different. We do not present a new proof system, security reduction, or signature construction. Instead, we ask a systems question:

How should agent platforms deploy behavior-constrained authorization so that it becomes operationally meaningful in real tool-using systems?

The answer developed here is architectural and control-theoretic. We argue that the right deployment pattern is a closed loop:

```
structured action -> policy-bound authorization -> validator gate ->
structured feedback -> safe retry
```

In other words, this paper should be read as a systems framing and deployment thesis built on top of the cryptographic direction opened by Li's work.

1.2 Contributions

This paper makes four contributions.

1. It presents a **systems framing** of behavior-constrained authorization for agent platforms, centered on structured actions, policy-bound authorization, validator-gated execution, and execution-path closure.
2. It introduces a **control-theoretic interpretation** of the architecture: the agent acts as controller, the validator acts as sensor and judge, and structured rejection reasons become error signals in a negative-feedback loop.

3. It argues that **task objectives and safety constraints are structurally similar** inside this loop. Both can be expressed as reference signals against which candidate actions are evaluated.
4. It offers a **deployment thesis**: the first useful applications do not require blockchain. Local or organizational validator-gated systems already capture much of the practical value.

1.3 Scope

This is a workshop-style systems position paper. It is not:

- a full cryptographic proof paper,
- a universal theory of agent safety,
- a claim that all policy enforcement should be cryptographic,
- or a claim that behavior-constrained mechanisms eliminate the need for software testing, monitoring, or human governance.

Its narrower claim is that high-risk machine-executable actions should no longer be left under ordinary unbounded credentials when a behavior-constrained alternative is available.

2. Positioning

The direct antecedent of this paper is **Y.Y.N. Li's** behavior-bound signature work [1]. Our paper differs in both objective and level of analysis. Li's contribution is cryptographic: it asks how behavioral constraints can become part of authorization validity itself. Our contribution is architectural: assuming that direction matters, how should real agent systems be organized so that constrained authorization becomes operationally meaningful?

The answer we develop is intentionally narrow. We treat behavior-constrained authorization as a systems primitive for high-risk agent actions, and we use a control-theoretic lens to explain why structured feedback, validator-gated execution, and independent observation materially improve the safety posture of delegated agent systems.

3. Problem Statement

The core problem is **delegated authority without bounded behavior**.

In current agent engineering, most protections live in the software layer:

- prompt instructions,
- middleware filters,
- workflow checks,
- API gateway rules,
- policy engines,
- approval systems,
- sandbox settings,
- and environment-specific access controls.

These layers matter, but they do not change the semantics of the credential itself. If an agent holds an ordinary signing key or an unrestricted API token, then the credential remains capable of authorizing arbitrary requests accepted by the downstream service. Policy is therefore external to authorization rather than intrinsic to it.

This creates a mismatch between human intent and machine enforcement. Humans think in constrained statements such as:

- "the agent may pay up to 200 USD to approved vendors,"
- "the agent may update staging feature flags but not production tables,"
- "the agent may delete temporary files in the sandbox but never system files,"
- "the agent may invoke deployment APIs only for staging,"
- "the agent may prepare an approval request but not finalize a high-value decision."

Ordinary credentials do not encode such statements. They merely authorize use. The practical question is therefore:

How should a real agent system enforce behavioral constraints at the action boundary, rather than relying entirely on soft software discipline after the fact?

4. Threat Model

We assume an agent participates in a larger execution environment and is authorized to request risky actions. The environment may include planners, tool adapters, payment connectors, database clients, filesystem accessors, deployment services, and enterprise workflow engines.

We consider the following threat classes.

4.1 Prompt Injection and Goal Hijacking

The agent receives instructions that redirect its behavior toward unauthorized recipients, endpoints, files, or environments.

4.2 Buggy Planning or Tool Use

The agent reasons incorrectly, misreads context, or selects a harmful tool invocation while still using syntactically valid credentials.

4.3 Runtime Replacement

The original agent process is replaced, modified, or wrapped by another runtime that retains access to the authorization mechanism.

4.4 Partial Toolchain Compromise

A plugin, tool wrapper, CI helper, or command adapter alters the final action after planning but before execution.

4.5 Unsafe Direct Invocation

The system exposes an execution path that accepts high-risk actions directly, bypassing policy validation.

This paper does not claim to solve:

- the truthfulness of external business claims,
- semantic quality evaluation,
- complete containment of arbitrary malware,
- or all forms of organizational fraud.

Its concern is narrower and more concrete: bounding which machine-executable actions a delegated agent can successfully authorize.

5. Position

The central position of this paper is:

Behavior-constrained authorization should be deployed as a hard control layer for high-risk agent actions.

This position has four immediate implications.

5.1 The Right Unit of Control Is the Structured Action

The system should not attempt to constrain free-form intent directly. It should constrain normalized action types such as:

- `payment`,
- `api_call`,
- `db_update`,
- `file_rm`,
- `deploy_start`,
- `approval_submit`.

The action is the unit at which policy becomes explicit, hashable, bindable, and machine-verifiable.

5.2 The Right Boundary Is the Authorization Boundary

If the system allows an ordinary signature or unrestricted token to be produced first, and asks whether the request was acceptable only afterward, then the credential has already remained too powerful. The authorization package should be produced only for requests that fit inside the permitted policy envelope.

5.3 The Validator Must Be the Only Gate to Execution

A behavior-constrained design collapses if the executor still exposes side channels, legacy endpoints, or administrative bypasses that avoid the validator.

5.4 Blockchain Is Not Required for First Deployment

The immediate value of behavior-constrained authorization appears in local or organizational control systems. Shared ledgers may matter later for auditability, decentralized ordering, or settlement, but they are not a prerequisite for the first practical safety gains.

6. Reference Architecture

We propose the following reference architecture.

```
Agent
  |
  v
Structured Action Builder
  |
  v
Policy-Bound Signer / Prover
  |
  v
Validator Gate
  |
  v
Execution Service
  |
  v
Audit / Monitoring / Feedback
```

Each component has a distinct role.

6.1 Agent

The agent plans, retries, and optimizes. It is productive, but not assumed to remain aligned under all conditions.

6.2 Structured Action Builder

This component converts raw intent into canonical machine-readable actions. It defines field layout, serialization rules, hashes, and allowed action types.

6.3 Policy-Bound Signer / Prover

This component is the hard boundary. It should emit an authorization package only for actions inside the allowed policy domain. In weaker MVP systems this may be a controlled signer. In stronger systems it may be a proof-producing component closer to the cryptographic model envisioned in Li's work.

6.4 Validator Gate

The validator checks:

- registered public key or delegated identity,
- action binding,
- policy or circuit version,
- proof or signature validity,
- replay protection,
- and contextual admissibility.

6.5 Execution Service

The executor performs the action only after the validator accepts it.

6.6 Audit and Feedback

The system records acceptance or rejection and emits structured reasons so that the agent can retry safely.

7. End-to-End Control Loop

The operational loop is:

```
Agent -> Action -> Sign/Prove -> Validate -> Reject or Execute -> Feedback ->
Retry
```

This loop matters for two reasons.

First, it changes the semantics of rejection. Rejection is not merely an exception; it is an input to the next round of control. The agent can inspect structured failure reasons such as:

- `amount_exceeds_limit`,
- `recipient_not_whitelisted`,
- `db_env_not_allowed`,
- `file_path_blocked`,
- `unknown_public_key`,
- `replayed_nonce`.

Second, it clarifies the distinction between hard and soft constraints.

- Hard constraints:
- machine-verifiable,
- deterministic,
- suitable for policy-bound authorization.
- Soft constraints:
- semantic,
- contextual,
- subjective,
- better handled by tests, review agents, evaluation pipelines, or humans.

Behavior-constrained authorization is strongest when used for hard boundaries, not soft judgment.

8. A Control-Theoretic Framing

The most important interpretive claim of this paper is that the above loop is not only an authorization pipeline. It is also a **negative-feedback control system** in the broad sense of control and feedback.

That framing matters because it explains why behavior-constrained authorization is valuable even before full cryptographic idealization. The architecture works not just by forbidding certain requests, but by separating control, observation, and execution in a way that produces safe iterative convergence.

8.1 Agent as Controller, Validator as Sensor

In a standard control loop, a controller proposes an output, a sensor measures the result against a reference condition, and a feedback channel returns error information. In the architecture described here:

- the **agent** is the controller,
- the **structured action** is the candidate output,
- the **policy and task boundary** provide the reference condition,
- the **validator** acts as sensor and judge,
- and the **rejection reason** is the error signal.

This framing shifts the role of the validator. The validator is not merely a gate that says yes or no. It is the part of the system that measures deviation from the allowed or desired region and returns that deviation in machine-usable form.

8.2 Structured Rejection Reasons as Error Signals

The architecture loses much of its value if rejection is reduced to a single opaque `denied`.

A useful validator should instead produce structured feedback such as:

```
{
  "status": "rejected",
  "reasons": [
    "amount_exceeds_limit",
    "recipient_not_whitelisted"
  ]
}
```

From a control-theoretic perspective, these reasons are error signals. They tell the controller where the current action deviates from the acceptable region. The richer the feedback, the more efficiently the agent can correct course.

8.3 Goals and Constraints Are Structurally Similar

One useful consequence of the control-theoretic framing is that **task goals** and **safety constraints** become structurally similar. Both act as reference signals.

Examples:

- "payment amount must not exceed 200 USD" is a boundary condition,
- "all tests must pass" is an acceptance target,
- "deployment must stay in staging" is an environment boundary,
- "risk score must remain below threshold" is a bounded target.

The system architecture does not need separate conceptual machinery for these cases. In each case, the validator measures deviation from a reference condition and returns feedback that guides the next attempt.

8.4 Separation Principle

Traditional agent systems often let the controller evaluate itself. The same agent that generates the plan also interprets whether it is safe, acceptable, or complete. From a control-theoretic perspective, this is structurally weak.

The relevant principle is a version of the **separation principle**: control and observation should not collapse into the same untrusted component. A behavior-constrained architecture improves this by moving measurement into an independent validator. The agent may plan freely, but it does not get to declare its own action valid.

This is one of the deepest practical insights gained from Li's underlying cryptographic direction. Behavior-aware authorization is not only about stronger signing. It is about enforcing an independent observation layer that the agent cannot redefine at will.

8.5 Stability Conditions

The control-theoretic framing also clarifies when the architecture will fail to converge.

The loop becomes weaker when:

- feedback is too coarse to guide correction,
- the controller cannot interpret the returned reasons,
- the target is unreachable under the available policy envelope,
- or the execution system contains bypass paths that create a competing positive-feedback route around the validator.

These are not merely implementation details. They are system-level stability conditions.

9. Action Modeling

Action modeling is the most underestimated step in the architecture.

Every high-risk operation must first be represented as a stable, structured action. For example:

9.1 Payment

```
{
  "type": "payment",
  "amount": 168.50,
  "currency": "USD",
  "recipient": "vendor_123",
  "invoice_id": "inv_20260311_001",
  "epoch": 202603111030,
  "nonce": "n-001"
}
```

9.2 API Invocation

```
{
  "type": "api_call",
  "service": "payment-gateway",
  "endpoint": "/v1/payouts",
  "method": "POST",
  "body_hash": "....",
  "epoch": 202603111035,
  "nonce": "n-002"
}
```

9.3 Database Update

```
{
  "type": "db_update",
  "env": "staging",
  "table": "feature_flags",
  "fields": ["enabled"],
  "where_scope": "id_eq",
  "row_limit": 1,
  "epoch": 202603111330,
  "nonce": "db-001"
}
```

9.4 File Removal

```
{
  "type": "file_rm",
  "path": "/workspace/sandbox/build/output.tmp",
  "recursive": false,
  "epoch": 202603111332,
  "nonce": "fs-001"
}
```

These examples are not claimed to be universal schemas. Their role is to illustrate the engineering principle: safety begins when the action space becomes explicit enough to bind, hash, validate, and reject deterministically.

10. Application Domains

The value of the architecture appears most clearly in domains where high-risk actions are already structured and policy-rich. The following sections are therefore not meant as an exhaustive catalog, but as evidence that the framing applies across several concrete classes of agent behavior.

10.1 Payments

Payments are one of the clearest application domains because the hard rules are usually explicit:

- amount cap,
- recipient whitelist,
- currency restriction,
- optional frequency or cumulative limits.

Under a behavior-constrained design:

1. the agent emits a structured payment action,
2. the signer or prover authorizes only policy-compliant actions,
3. the validator checks binding, policy version, and replay,
4. the executor forwards only accepted requests to the payment rail.

The gain is not that the payment stack becomes semantically perfect. Fake invoices, fraudulent business claims, and bad whitelist configuration remain possible. The gain is narrower but valuable: an agent with delegated authority no longer has unrestricted payment authority.

10.2 API Invocation

Many risky actions are API invocations in disguise. Useful constraints include:

- service whitelist,
- endpoint whitelist,
- method whitelist,
- resource identifier scope,
- body hash binding,
- quota and frequency limits.

This domain is especially practical because the validator can often be placed directly in front of an existing API adapter or gateway. That makes it a good example of why blockchain is unnecessary for first deployment.

10.3 Autonomous Software Development

Autonomous software development is where many readers first think of agent safety, but it is also where category errors are common. Behavior-constrained authorization should not be treated as a judge of code elegance, design taste, or product judgment. Those are soft evaluation problems.

The natural fit is dangerous machine action:

- `db_update`,
- `file_rm`,
- `shell_exec`,
- `deploy_start`.

Useful constraints include:

- no production database writes,
- no writes to non-whitelisted tables,
- no file removal outside `/workspace/sandbox/**`,
- no deletion under `/etc`, `/usr`, or `/bin`,
- deployment only to staging,
- no access to production secrets.

This is the domain where the distinction between hard and soft constraints becomes especially important. The system does not need to prove that the code is good. It needs to prevent the agent from deleting the wrong files, updating the wrong tables, or deploying to the wrong environment.

10.4 Enterprise Approval and Finance

Enterprise workflow systems are another natural fit because approval logic often contains explicit thresholds and scopes:

- maximum approvable amount,
- allowed approval category,
- vendor whitelist,
- role restrictions,
- document presence requirements,
- per-period cumulative limits.

Here the value is not simply automation. It is that the authority boundary moves closer to the credential itself. A constrained approval identity can be prevented from authorizing categories or amounts outside its policy envelope.

11. Key Custody and Identity Binding

A practical system must decide how the agent obtains authorization capability. Three models are common.

11.1 Agent Holds a Restricted Capability Directly

This is the simplest prototype model and usually the least safe production model.

11.2 Agent Calls a Controlled Signer

The agent does not access raw secret material. It submits a structured action to a signer or prover service that emits an authorization package only if the policy is satisfied.

This is the preferred production posture for many systems.

11.3 Agent Uses Short-Lived Delegated Identities

The agent is issued a scoped identity limited by time, purpose, or environment. This reduces blast radius and simplifies rotation.

In all cases, the validator must know which public keys or identity commitments are authorized for which action domains. Requests from unregistered keys should be rejected

immediately.

12. Why Blockchain Is Not Required by Default

There is a common temptation to treat behavior-constrained authorization as immediately implying a new ledger or blockchain design. That is unnecessary for first deployment.

For payments, API calls, software delivery pipelines, database safety, and enterprise approvals, a local or organizational validator already delivers substantial value. The key gains come from:

- structured action modeling,
- constrained authorization,
- replay protection,
- validator gating,
- and auditable rejection reasons.

A blockchain becomes relevant only when the application additionally requires:

- multi-party distrust,
- public auditability,
- decentralized ordering,
- or shared settlement.

The practical deployment order should therefore usually be:

1. local hard-control layer first,
2. shared ledger later if needed.

13. Deployment Considerations

A behavior-constrained system becomes useful only when several operational details are handled correctly.

13.1 Replay Protection

Every action package needs replay protection through nonce design, epoch binding, and validator-side nonce storage.

13.2 Policy Versioning

Policies inevitably change. The authorization package must therefore bind not only the action, but also the policy or circuit version against which it was evaluated.

13.3 Structured Failure Reasons

The system should return machine-readable rejection causes so that the agent can retry safely. Without structured feedback, the architecture loses much of its control advantage.

13.4 Execution-Path Discipline

No side channel should be able to perform a high-risk action without passing through the validator.

13.5 Auditability

The system should record:

- action type,
- action hash,
- identity,
- policy version,
- validator decision,
- reason codes,
- and execution outcome.

14. Limits and Objections

This paper deliberately makes a narrower claim than "all agent safety can be solved cryptographically." Several objections and open problems remain important.

14.1 Soft Judgment Remains Outside the Core Mechanism

Many important questions are not naturally expressible as bounded machine predicates. Whether a patch is elegant, a design choice is wise, or a business plan is persuasive is not the right target for behavior-constrained authorization.

14.2 Stateful Constraints Are Harder Than Local Constraints

Rate limits, cumulative budgets, and historically dependent rules require stronger state binding and are harder than purely local checks.

14.3 Proof Systems Remain Operationally Expensive

A full proof-producing system introduces proving cost, verification cost, circuit management, and deployment complexity. The systems framing presented here does not erase that difficulty.

14.4 Compatibility Is Limited

Many existing services accept only ordinary signatures or ordinary API credentials. Stronger behavior-constrained authorization often requires control over the execution gateway.

14.5 Bypass Risk Still Dominates in Practice

Even a strong authorization mechanism fails if the executor still accepts a legacy, direct, or administrative path.

14.6 This Is Not a Claim of Universal Safety

The architecture reduces over-authorization. It does not prove business truth, prevent every social attack, or eliminate the need for testing, auditing, monitoring, and human governance.

15. Discussion

The strongest reason to care about behavior-constrained authorization today is not that full cryptographic perfection has already arrived. It is that the architectural pattern is already useful.

Even in weaker MVP forms, the pattern changes the system:

- actions become explicit,
- authority becomes bounded,
- validators become first-class infrastructure,
- failures become structured,
- and risky execution paths become easier to reason about.

That matters because many real agent failures are not cryptanalytic failures. They are failures of coarse authorization. Agents are given capabilities far broader than their task envelope because ordinary credentials are too blunt. The systems contribution of this paper is to argue that behavior-constrained authorization should be treated as the missing hard boundary in those environments.

The control-theoretic framing sharpens that thesis. It explains why the validator is not just a blocking mechanism, but an observation mechanism. It explains why structured reasons matter, why retry can be safe rather than ad hoc, and why independent validation is superior to agent self-evaluation. Most importantly, it shows that the system is not merely preventing bad actions; it is shaping the dynamics of how an agent approaches acceptable ones.

This is where the connection to **Y.Y.N. Li's** reference paper becomes especially important. Li's work points toward authorization that is behavior-aware at the cryptographic level. Our claim is that, once taken seriously as a systems primitive, that idea naturally leads to a control architecture for safe delegated agency.

16. Conclusion

This paper argued that behavior-constrained authorization should be understood as more than a stronger signature primitive. Building on the behavior-bound signature direction introduced by **Y.Y.N. Li**, we argued that it should be deployed as a **hard, negative-feedback control layer** for high-risk agent actions.

The practical pattern is:

```
structured action -> policy-bound authorization -> validator gate ->
execution -> structured feedback -> retry
```

When implemented correctly, this pattern allows agents to search, retry, and optimize inside a bounded action space while preventing them from unilaterally authorizing actions outside that space.

The strongest immediate applications are not soft semantic judgments, but hard machine actions such as payments, API calls, database mutations, file deletions, deployments, and scoped approvals. These are the domains where behavior-constrained authorization can provide immediate safety value without requiring a blockchain-first deployment strategy.

The broader claim of this paper is therefore modest but concrete:

safe agent engineering improves when authorization becomes behavior-aware, action-structured, validator-gated, and embedded in an independent feedback

loop.

References

[1] **Y.Y.N. Li.** *Behavior-Bound Signatures: Cryptographic Policy Enforcement from Zero-Knowledge Soundness*. Zenodo, 2025. DOI: `10.5281/zenodo.18811273`. Available at: https://doi.org/10.5281/zenodo.18811273.